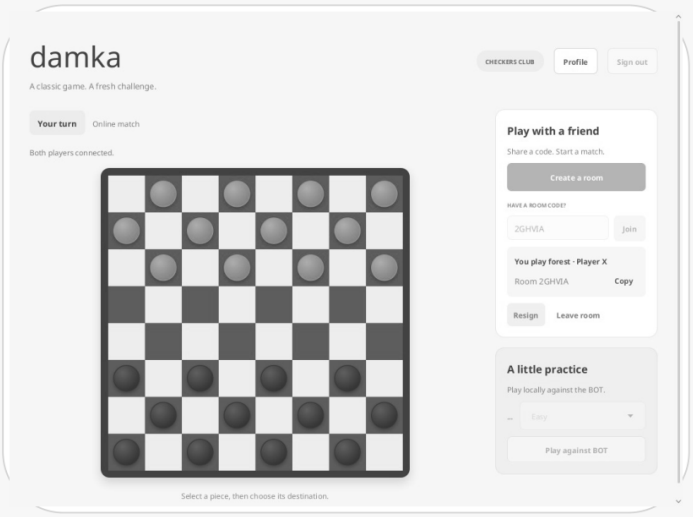
Damka

Checkers, connected.

TCP multiplayer · Local BOT
Accounts, statistics and history

Muthanna Farhat
Rabea Abu Aad
Supervisor: Inbal Ronen



Slide 1

One game, two ways to play

Keep both screens in sync — and remember the result.

Online matches

Two computers
One room code

Local BOT

Three difficulty levels
Offline practice

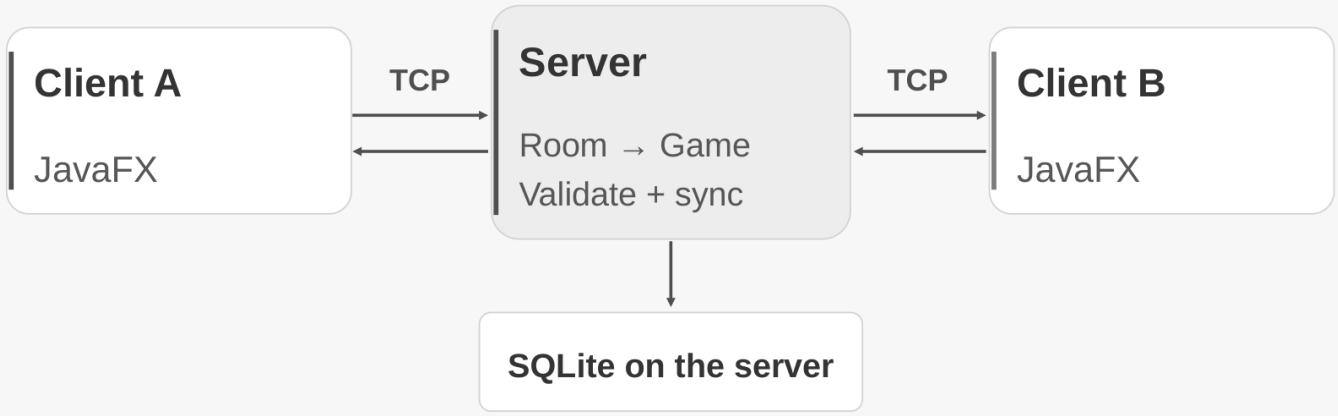
Personal profiles

Statistics
Match history

One shared rules engine: Game

Slide 2

Why a client–server design?

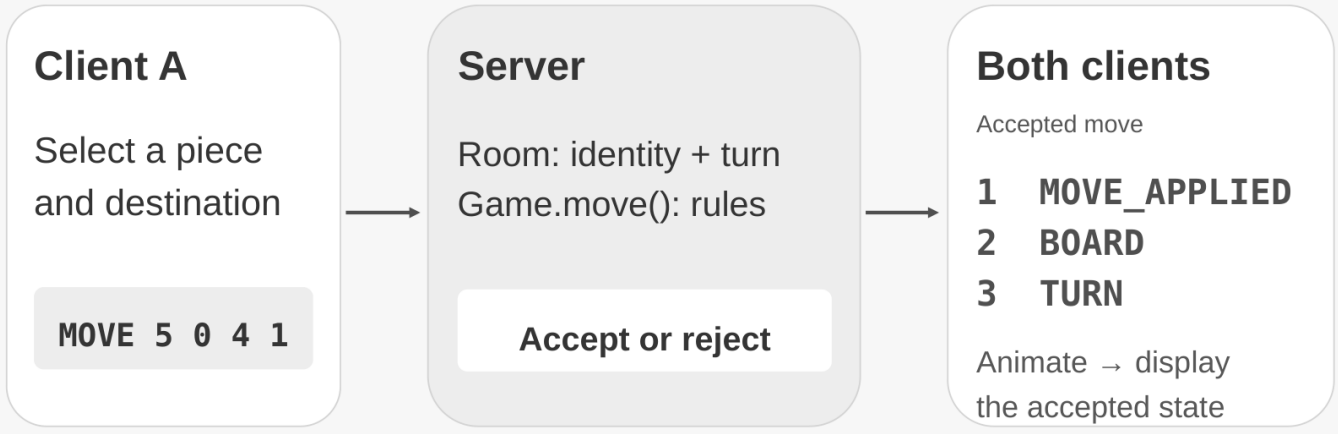


Choice: persistent TCP for two-way updates

Alternative: HTTP polling

Slide 3

From a click to both screens



UTF-8 · one message per line · network work off the UI thread

Slide 4

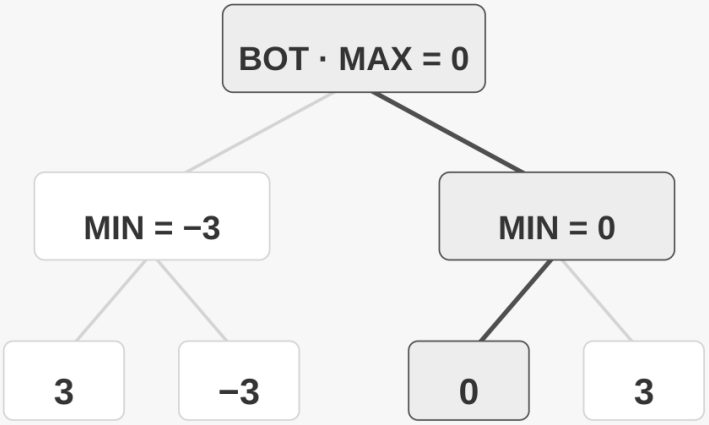
Minimax: how the BOT chooses

Shared game rules

Legal moves on copies

Piece = 3 King = 6

Depth: 1 / 2 / 3



Illustrative two-step search

Alpha–Beta skips branches that cannot improve the decision.

Slide 5

Accounts that remember results

Player profile

Illustrative account

ONLINE	BOT
1 win / 1 match	0 matches

Match history

Online	WIN	Resignation

`profiles`

Username · salt · hash

`game_results`

Players · result · mode · time

Saved results → statistics

Slide 6

Verification and AI-assisted development

48 **automated tests**

Game rules and edge cases
Real TCP protocol checks
SQLite persistence and deduplication

Documented passing run: 12 Oct 2026

Codex + GitHub Copilot

Planning · code · explanations
Tests and documentation

Read → run → review → refine

Slide 7

Current limits and next step

No match resume

A detected disconnect
ends the active match

Local BOT results

Client-reported
No offline upload queue

NEXT **In-room chat over the existing TCP connection**

Slide 8

What we built and learned

One authoritative game.
Two synchronized clients.
Persistent player results.

Built to play.

Rules · Networking
BOT · Profiles

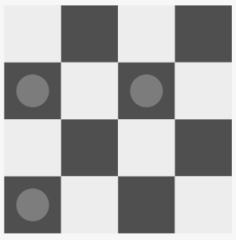
Rabea Abu Aad — Game Logic
Muthanna Farhat — TCP Communication

github.com/MuthannaFrhat/damka-game

Slide 9

THANK YOU

Questions & discussion



github.com/MuthannaFrhat/damka-game

Slide 10